

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет прикладної математики та інформатики
Кафедра математичного моделювання соціально-економічних процесів

Звіт

Лабораторна робота №3

Виконав:

студент групи Пма-42

Думанський Юрій

Перевірила:

доцент Лисецька О. Ю.

Львів-2026

Тема:

Робота з файловою системою та автоматизація резервного копіювання засобами мови Python

Завдання:

1. На базі програми щодо отримання даних про розміри файлів та підкаталогів реалізувати виведення результатів у файл 'dirslist.txt', перенаправивши стандартне виведення у цей файл і створивши для нього окремий каталог.
2. Використовуючи можливості модуля shutil, реалізувати backup з кількох каталогів одночасно.

Хід роботи:

- Для виконання першого завдання було використано програмну основу для обчислення розміру директорій. Основна функція dirsize за допомогою os.walk рекурсивно обходить дерево каталогів, підсумовуючи об'єм усіх знайдених файлів за формулою: $S = \sum_{i=1}^n size(f_i)$, де n — кількість файлів у підкаталогах.
- За допомогою функції os.mkdir створено окрему директорію для зберігання звітів. Реалізовано механізм перенаправлення стандартного потоку виведення sys.stdout у файл dirslist.txt, що дозволило автоматично зберегти відсортований список пар (каталог, розмір) без зміни основної логіки викликів print().
- У другому завданні проведено оптимізацію скрипта для резервного копіювання. Замість побайтового читання файлів вручну використано високорівневі інструменти модуля shutil. Зокрема, функція shutil.copytree забезпечує рекурсивне копіювання всієї структури папок, а shutil.rmtree дозволяє швидко видаляти застарілі дані перед архівацією.
- Програма підтримує обробку кількох джерел одночасно. Назви цільових каталогів генеруються на основі поточного часу за допомогою datetime.strftime, що запобігає перезапису попередніх версій та забезпечує контроль версій у форматі РікМісяцьДень_ГодинаХвилинаСекунда.
- Код програми наведений в додатку 1.

Результати:

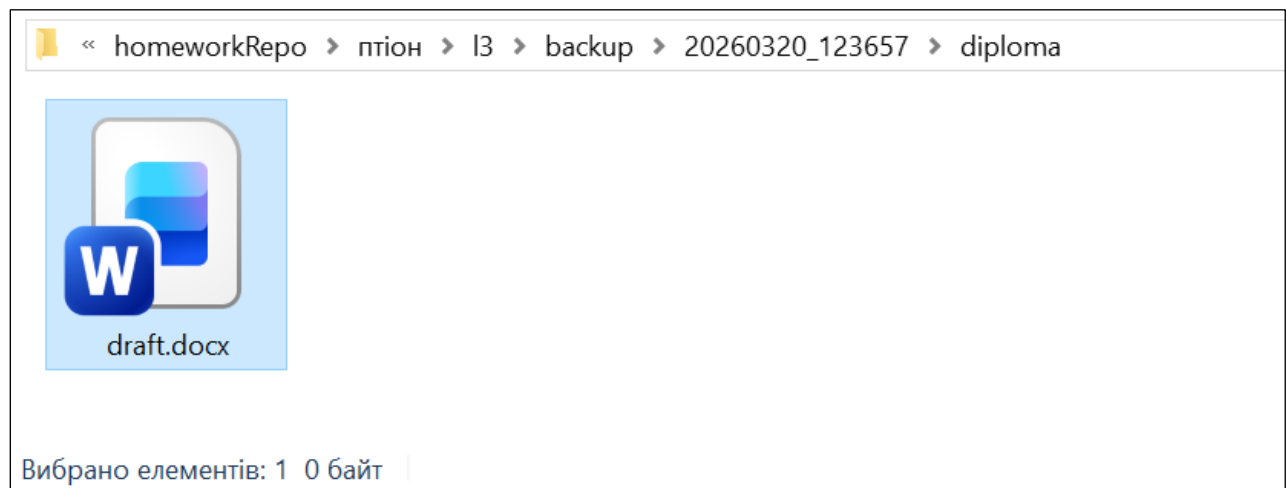
Програма успішно створює звіт про розміри папок у новому каталозі, коректно перенаправляючи дані у текстовий файл.

```
F:\дз-----\homeworkRepo\птіон\l3>1.py
Input directory name: ..
Готово! Результати збережено у: output_results\dirslist.txt
```

1	directory: ..\l1, size: 70528528
2	directory: ..\l2, size: 629451
3	directory: ..\ind1, size: 387140
4	directory: ..\lk1, size: 69963
5	directory: ..\l3, size: 10704

Система бекапу виконує копіювання списку заданих директорій у спільну папку з унікальним штампом часу, попередньо архівуючи старі записи у форматі .tar.gz.

```
F:\дз-----\homeworkRepo\птіон\l3>2.py
Введіть backup-каталог: backup
Введіть імена каталогів для бекапу через кому:
..,"F:\дз-----\diploma"
Пропущено директорію ...
[WinError 183] Cannot create a file when that file already exists: 'backup\20260320_123657\..'
Скопійовано: F:\дз-----\diploma -> backup\20260320_123657\diploma
```



Висновки:

Під час лабораторної роботи було освоєно автоматизацію рутинних операцій із файловою системою. Використання бібліотеки shutil значно скоротило об'єм коду та підвищило надійність процесу копіювання у порівнянні з низькорівневими методами. Навички перенаправлення потоків виводу дозволяють гнучко керувати звітами в автоматизованих системах.

Додаток 1. Код програми.

```
import os
import sys

def dirsizе(path):
    """Функція, що повертає розмір однієї директорії (каталогу)"""
    size = 0
    for root, dirs, files in os.walk(path):
        size += sum(os.path.getsize(os.path.join(root, name)) for name in files)
    return size

def dirslist(directory):
    """Функція, що повертає список пар (каталог, розмір)"""
    dir_list = os.listdir(directory)
    dirs_sizes = []

    for dir in dir_list:
        path = os.path.join(directory, dir)

        if os.path.isdir(path):
            dirs_sizes.append((path, dirsizе(path)))

    dirs_sizes.sort(reverse=True, key=lambda x: x[1])
    return dirs_sizes

if __name__ == '__main__':
    if len(sys.argv) == 1:
        directory = input('Input directory name: ')
    else:
        directory = sys.argv[1]

    dirs_sizes = dirslist(directory)

    # 1. Створюємо окремий каталог для файлу
    output_dir = "output_results"
    if not os.path.exists(output_dir):
        os.mkdir(output_dir)

    output_file_path = os.path.join(output_dir, 'dirslist.txt')

    # 2. Зберігаємо оригінальний стандартний вивід
    original_stdout = sys.stdout

    # 3. Перенаправляємо вивід у файл
    with open(output_file_path, 'w', encoding='utf-8') as f:
        sys.stdout = f

        # Усі ці принти тепер записуватимуться безпосередньо у dirslist.txt
        for _ in dirs_sizes:
            print(f'directory: {_[0]}, size: {_[1]}')

    # 4. Відновлюємо стандартний вивід у консоль
    sys.stdout = original_stdout
    print(f"Готово! Результати збережено у: {output_file_path}")
```

```

import os
import datetime
import sys
import tarfile
import shutil

def archivesubdirs(backupdir):
    """Функція для архівації всіх підкаталогів та їх подальшого видалення"""
    backupdir = os.path.normpath(backupdir)

    # Якщо каталогу бекапів ще немає, створюємо його і виходимо
    if not os.path.exists(backupdir):
        os.makedirs(backupdir, exist_ok=True)
        return

    lst = os.listdir(backupdir)

    for name in lst:
        full_name = os.path.join(backupdir, name)

        if os.path.isdir(full_name):
            try:
                # Архівуємо
                with tarfile.open(full_name + '.tar.gz', 'w:gz') as tf:
                    # arcname=name гарантує, що в архіві буде папка, а не повний шлях C:\...
                    tf.add(full_name, arcname=name)

                # Використовуємо shutil для миттєвого видалення каталогу з усім вмістом
                shutil.rmtree(full_name)
            except Exception as e:
                print('Помилка архівації/видалення', name, e, sep='\n')

def getbackupname(backupdir):
    """Функція, що повертає ім'я нового каталогу з поточної дати та часу."""
    dt = datetime.datetime.now()
    dirname = dt.strftime('%Y%m%d_%H%M%S')
    return os.path.join(backupdir, dirname)

def backupdirectories(directories, backupdir):
    """Функція, що зберігає файли із зазначених каталогів у каталозі backup"""
    archivesubdirs(backupdir) # Архівуємо попередні версії

    toparent = getbackupname(backupdir)
    os.makedirs(toparent, exist_ok=True)

    for src_dir in directories:
        src_dir = os.path.normpath(src_dir)

        if not os.path.exists(src_dir):
            print(f'Каталог "{src_dir}" не знайдено. Пропускаємо.')
            continue

        last_dir_name = os.path.split(src_dir)[-1]
        dest_dir = os.path.join(toparent, last_dir_name)

        try:

```

```

        # shutil.copytree автоматично створює цільову директорию і копіює все туди
        shutil.copytree(src_dir, dest_dir)
        print(f'Скопійовано: {src_dir} -> {dest_dir}')
    except Exception as e:
        print(f'Пропущено директорию {src_dir}.\n', e)

if __name__ == '__main__':
    if len(sys.argv) < 3:
        backupdir = input('Введіть backup-каталог: ').strip(' \\'')
        user_input = input("Введіть імена каталогів для бекапу через кому:\n")
        directories = [d.strip(' \\'') for d in user_input.split(",") if d.strip()]
    else:
        backupdir = sys.argv[1]
        directories = sys.argv[2:]

    backupdirectories(directories, backupdir)

```